



# Data Structures and Algorithms -3 (25CS2103E)

## Pattern Matching using Z-Algorithm

### Team Members

2520030141 – Rayanki Himavarshini

2520030514 – N. Sai Pranavi

2520030587 – J. Sai Spoorthi

Under the Esteemed Guidance of

**Anuradha, Department of CSE, KLH  
University**

# Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Flow Chart
- Results
- Hardware and Software requirements
- Conclusion
- Future Scope

# Abstract

Pattern matching is the process of locating every occurrence of a small string (the pattern) within a larger string (the text). It underlies search engines, text editors, DNA sequencing tools, and cyber-security systems. The naive character-by-character approach re-examines text after every failed attempt, giving a worst-case time complexity of  $O(n \times m)$  that degrades sharply on large or repetitive data.

This project implements the Z-Algorithm, a linear-time exact string-matching technique. The pattern and text are concatenated into a single string  $S = P + \$ + T$ , and a Z-array is built in one pass, where  $Z[i]$  is the length of the longest substring starting at position  $i$  that matches a prefix of  $S$ . A pattern occurrence is found wherever  $Z[i]$  equals the length of the pattern.

By reusing previously matched information through the Z-box technique, the algorithm avoids redundant comparisons and guarantees  $O(n + m)$  time in the best, average, and worst cases. The system is implemented in Java and validated on sample text-pattern pairs, including overlapping and near-miss cases, to confirm correctness and demonstrate its efficiency over naive searching.

## What is Pattern Matching?

Pattern matching is the process of finding all occurrences of a smaller string, called the pattern, within a larger string, called the text. It is one of the most fundamental problems in computer science and forms the backbone of search, retrieval, and analysis systems.

*Example: In the text "...ABCABCABY...", the pattern "CABCABY" is located at a specific starting index within the text.*

## Real-World Applications

- Search Engines – locating keywords across billions of web pages
- Text Editors – powering find, find-and-replace, and syntax highlighting
- DNA Sequencing – matching genetic sub-sequences in bioinformatics
- Cyber Security – intrusion detection and malware signature scanning

# Literature Survey

Existing pattern-matching techniques were reviewed to identify a suitable linear-time approach for this project:

## Naive (Brute-Force) Search

- Slides the pattern over the text one position at a time. Simple to implement but re-compares characters after every mismatch, giving  $O(n \times m)$  worst-case time (Cormen et al., Introduction to Algorithms, 3rd Ed.).

## Knuth–Morris–Pratt (KMP) Algorithm

- Preprocesses the pattern into a failure (LPS) function to skip redundant comparisons, achieving  $O(n + m)$  time (Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press).

## Z-Algorithm

- Builds a single Z-array over the concatenated pattern and text, fully reusing prior comparisons to guarantee  $O(n + m)$  time with a simpler implementation than KMP (Landau & Vishkin, Fast String Matching with k Differences, JCSS).

Across the literature, the Z-Algorithm is consistently cited as a linear-time alternative that is easier to implement while matching KMP's efficiency — motivating its choice for this project.

# Problem Statement and Objectives

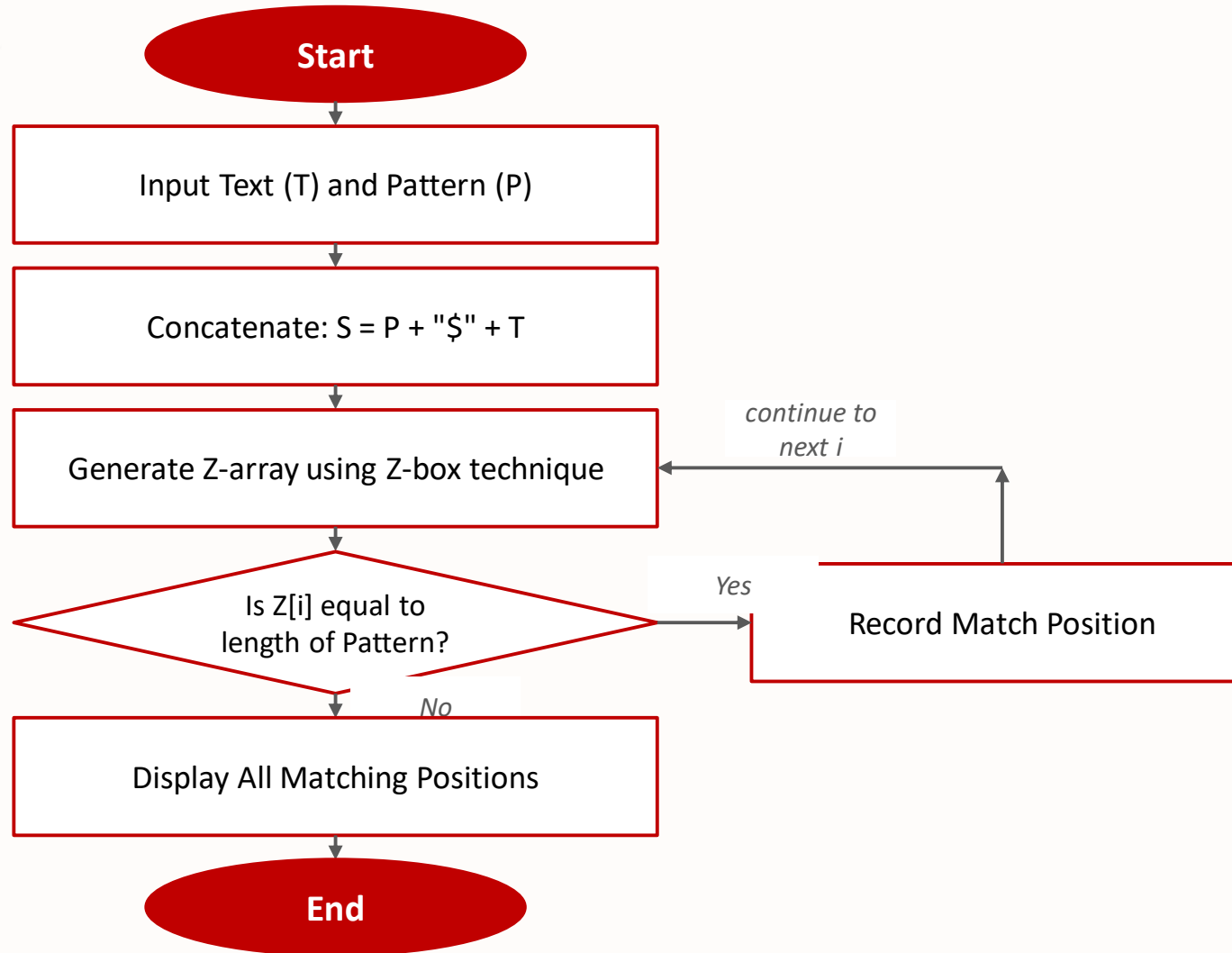
## Problem Statement

Given a text  $T$  of length  $n$  and a pattern  $P$  of length  $m$ , find all starting indices in  $T$  where  $P$  occurs. As texts grow larger — web pages, genomes, log files — the searching method used has a direct impact on speed and system responsiveness. Naive character-by-character comparison re-examines the same text repeatedly, degrades to  $O(n \times m)$  on repetitive inputs, and does not scale to large real-world data.

## Objectives

- Efficient Pattern Detection – find every occurrence of a pattern inside a text quickly and reliably, regardless of text size.
- Reduce Unnecessary Comparisons – reuse previously computed match information instead of re-scanning characters already examined.
- Improve Search Performance – achieve linear  $O(n + m)$  time complexity so performance scales predictably with input size.

# Flow Chart



# Results

## Sample Run

Text: ABCDABCDABEABCDABD

Pattern: ABCD

Output: Pattern found at index 0, index 4, and index 14 – Total occurrences: 3

## Observations

- The overlapping occurrence at index 4 is still detected correctly.
- The mismatch region around index 8–9 (“AB E”) is correctly skipped.
- All three genuine occurrences of “ABCD” are reported with zero missed or false matches.

## Time Complexity

The Z-Algorithm achieves  $O(n + m)$  time in the best, average, and worst cases, with  $O(n + m)$  auxiliary space — unlike naive search, there is no worst-case blow-up on repetitive inputs.

# Hardware and Software requirements

## Hardware Requirements

- Processor: Intel Core i3 or above (or equivalent)
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 20 GB free space
- Standard keyboard, mouse, and display

## Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Java (JDK 8 or above)
- IDE: Eclipse, IntelliJ IDEA, or VS Code
- Any standard text editor for viewing sample input/output

## Conclusion

The Z-Algorithm solves exact pattern matching in guaranteed  $O(n + m)$  time by building a single Z-array over the concatenated string Pattern + "\$" + Text. It outperforms naive search and matches the efficiency of KMP with a simpler design that needs only one array and a few pointers.

Implementing this project gave practical understanding of string-processing algorithms and preprocessing techniques, hands-on experience building linear-time algorithms in Java, and deeper insight into algorithmic complexity analysis and trade-offs between different string-matching approaches.

Overall, the project meets its objectives of efficient, reliable, and scalable pattern detection, and provides a solid foundation for extending the work to more advanced search applications.

## Future Scope

- GUI Application – build a desktop or web interface so users can search visually instead of via console input.
- Multiple Pattern Search – extend the algorithm with generalized structures to search for many patterns at once.
- File Search Integration – apply the algorithm to search across entire files or directories, not just single strings.
- Real-Time Search – support incremental / streaming text so results update live as new data arrives.
- AI-Based Text Analytics – combine pattern matching with NLP techniques for smarter, context-aware text analysis.



**Thank you**